

Tutorial - Semana 6 - Encontro 1

Introdução

Até a Semana 5, o Vertical Slice ganhou comunicação desacoplada: `BPI_Interactive` permite que qualquer Actor seja chamado genericamente pelo `InteractionComponent`, e o Event Dispatcher permite que esse Actor avise sua própria reação sem conhecer quem escuta. Falta resolver um problema diferente: onde mora a informação sobre *o que* está sendo coletado ou ativado — nome, descrição, ícone de um item. Se essa informação ficar hardcoded dentro de cada Actor, qualquer ajuste de design exigirá abrir Blueprints individuais, e nenhum sistema futuro (como o Inventário da Semana 10) poderá consultá-la de forma genérica. Este encontro constrói `DT_Items`, a Data Table que centraliza essa informação, separando dado de design de lógica de gameplay.

Este tutorial não substitui a explicação do professor em sala. Ele existe para que você possa acompanhar a implementação passo a passo durante o laboratório e revisar os passos depois da aula, sem depender da documentação oficial da Epic.

Objetivos da Semana

- Explicar Data Tables e Data Assets como mecanismos de separação entre dado de design e lógica de gameplay.
- Comparar Data Table/Data Asset com ScriptableObject na Unity.
- Criar `DT_Items` como tabela centralizada de itens do Vertical Slice.

Resultado Esperado ao Final da Semana

`DT_Items` criada na subpasta `Data/DataTables/`, com nomenclatura conforme `PROJECT_ARCHITECTURE.md` e ao menos uma linha de exemplo preenchida, pronta para receber tipagem via Struct e Enum no Encontro 2.

Pré-requisitos

- `BPI_Interactive` implementada em ao menos um Actor do nível de teste (Semana 5).
 - Event Dispatcher de interação funcional (Semana 5, Encontro 2).
-

Antes de começar

O que você deverá possuir antes desta semana

- O projeto da Semana 5, com o objeto interativo funcional (`BPI_Interactive` + Event Dispatcher) integrado ao Vertical Slice.

Arquivos necessários

- Nenhum arquivo externo é necessário neste encontro.

Assets utilizados

- Nenhum asset novo obrigatório. Ícones de itens podem usar texturas simples já disponíveis no projeto ou placeholders (formas coloridas), já que a qualidade artística não é critério de avaliação.

Projeto esperado

O projeto da Semana 5, com o objeto interativo (porta, alavanca ou equivalente) funcional no nível de teste.

Parte 1 — Data Tables e Data Assets: separando dado de lógica

Objetivo

Compreender a diferença entre Data Table e Data Asset e criar `DT_Items` como tabela centralizada de itens do Vertical Slice.

Conceito

Até aqui, todo comportamento do Vertical Slice foi definido por lógica dentro de Blueprints. Mas nem toda informação de um jogo é comportamento — muita coisa é apenas *dado*: o nome de um item, sua descrição, seu ícone. Se essa informação estiver fixada dentro do Blueprint do próprio Actor, dois problemas aparecem. Primeiro, qualquer ajuste de design exige abrir e editar Blueprints individuais, um por um. Segundo, nenhum sistema futuro — como um Inventário — pode consultar esses dados de forma genérica, porque eles não existem em um lugar central.

Toda engine madura resolve isso com uma camada de dados independente da lógica que a consome. A Unreal oferece duas ferramentas complementares para isso:

- **Data Table:** uma estrutura tabular, editável como planilha, onde cada linha representa uma instância de dado. É a ferramenta correta quando existem várias instâncias com os mesmos atributos — por exemplo, uma coleção de itens.
- **Data Asset:** um asset independente que representa um único dado, mais complexo ou que não se organiza bem em linhas e colunas.

Nesta aula, o foco é a Data Table, porque `DT_Items` precisa representar uma coleção de itens com os mesmos atributos (nome, descrição, ícone).

Passo a passo

1. No Content Browser, navegar até a subpasta `Data/DataTables/` (criar essa subpasta caso ainda não exista, conforme a estrutura de `Content/` descrita em `PROJECT_ARCHITECTURE.md`).
2. Clicar com o botão direito na subpasta e selecionar **Miscellaneous > Data Table**.
3. Ao ser solicitada a escolha da estrutura de linha (Row Structure), selecionar a estrutura padrão do editor por enquanto (a estrutura customizada será criada no Encontro 2).
4. Nomear o asset `DT_Items`, conforme a convenção `DT_` de `PROJECT_ARCHITECTURE.md`.
5. Abrir `DT_Items` com duplo clique e adicionar de duas a três linhas de exemplo, preenchendo colunas básicas: nome, descrição e caminho do ícone.
6. Salvar `DT_Items`.

Resultado esperado

`DT_Items` criada na subpasta `Data/DataTables/`, com ao menos uma linha de exemplo preenchida, existindo de forma independente de qualquer Actor específico do Vertical Slice.

Verificando

Abrir `DT_Items` no Content Browser e confirmar que as linhas cadastradas aparecem corretamente no Data Table Editor, com os valores preenchidos em cada coluna.

Problemas comuns

- **Recriar a mesma informação em variáveis dentro do Actor de coleta:** se o nome ou a descrição de um item também existir como variável fixa dentro de um Blueprint, a informação está duplicada e desatualizável em um único lugar. Pergunte a si mesmo: se este item aparecer em três baús diferentes, quantos lugares precisariam ser editados? Se a resposta for mais de um, o dado pertence à tabela, não ao Actor.
- **Criar `DT_Items` solta na raiz de `Content/`:** toda Data Table deve estar na subpasta temática correta (`Data/DataTables/`), conforme `PROJECT_ARCHITECTURE.md`.
- **Deixar o nome padrão do editor (`NewDataTable`):** renomear sempre para `DT_Items` , nunca manter o nome gerado automaticamente.

Boas práticas

Trate `DT_Items` como a única fonte de verdade sobre os itens do Vertical Slice. Nenhum Actor deve armazenar, em variáveis próprias, informação que já existe em uma linha da tabela.

Comparação com Unity

ScriptableObject resolve o mesmo problema de dado desacoplado da lógica, mas por um caminho arquitetural diferente: cada instância de ScriptableObject é um asset independente, e uma coleção de itens normalmente exige que o desenvolvedor organize manualmente uma lista ou array desses assets. A Data Table da Unreal já nasce orientada a coleção — uma única planilha com uma linha por item — o que a torna mais direta para dados homogêneos em grande quantidade, enquanto o ScriptableObject é mais direto para um dado único e complexo com lógica própria embutida. O princípio — dado como asset, não como variável hardcoded em um Blueprint — é o mesmo nas duas engines.

Ao final da semana

Ao final da Semana 6 (Encontros 1 e 2), o Vertical Slice deve possuir `DT_Items` funcional, tipada por `S_ItemData` e `E_ItemType` , populada com um conjunto próprio de itens coletáveis, e um Actor de

coleta integrado que reutiliza `BPI_Interactable` e o Event Dispatcher da Semana 5 para sinalizar a coleta. Este encontro entrega apenas a primeira metade desse resultado: a tabela existe, mas ainda sem tipagem forte — isso é resolvido no Encontro 2. Corresponde ao início da linha "DT_Items (Data Table + Struct + Enum)" do roadmap descrito em PROJECT_ARCHITECTURE.md (seção 6, Módulo 2).

Desafio

Não há desafio de liberdade de solução neste encontro — a criação da tabela é demonstração e adaptação guiada, preparando a base de dados para o desafio do Encontro 2, quando cada grupo modelará seu próprio conjunto de itens.

Checklist

- ☐ Subpasta `Data/DataTables/` criada conforme PROJECT_ARCHITECTURE.md
- ☐ `DT_Items` criada com nomenclatura correta (`DT_`)
- ☐ Ao menos uma linha de exemplo preenchida (nome, descrição, ícone)
- ☐ Nenhuma informação de item duplicada em variáveis de Actor
- ☐ `DT_Items` salva e visível no Data Table Editor

Glossário

- **Data Table:** estrutura tabular de dados da Unreal, onde cada linha representa uma instância de um mesmo tipo de dado, editável como planilha dentro do editor.
- **Data Asset:** asset independente que representa um único dado, usado quando a informação não se organiza bem em linhas e colunas de uma Data Table.
- **Row Structure:** o tipo (Struct) que define as colunas de uma Data Table; neste encontro ainda é o tipo padrão do editor, substituído por `S_ItemData` no Encontro 2.

Referências

- EPIC GAMES. **Unreal Engine 5 Documentation** — Gameplay Framework in Unreal Engine (Data Assets e Data Tables). Disponível em: <https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-framework-in-unreal-engine>.
- EPIC GAMES. **Unreal Engine Learning Library** — introdução a Data Tables. Disponível em: <https://dev.epicgames.com/community/unreal-engine/learning>.

- UNITY TECHNOLOGIES. **Unity Manual** — ScriptableObject, para fins comparativos. Disponível em: <https://docs.unity3d.com/Manual/>.
- Vídeo sugerido (apoio complementar, nunca substituindo a documentação oficial): canal oficial **Unreal Engine**, vídeos introdutórios de Data Tables; **Mathew Wadstein**, explicação pontual de WTF Is? Data Table.

Imagem sugerida

Objetivo: mostrar a diferença conceitual entre Data Table e Data Asset. Enquadramento: diagrama lado a lado, dois blocos. Elementos importantes: à esquerda, uma planilha estilizada com várias linhas rotuladas "Item 1", "Item 2", "Item 3" representando a Data Table; à direita, um único ícone de asset isolado representando o Data Asset. O que deve ser destacado: a Data Table concentra várias instâncias homogêneas em um único asset; o Data Asset representa uma instância única e independente. Legenda sugerida: "Data Table: coleção tabular de itens homogêneos — Data Asset: instância única e independente."